

ATIVIDADE
- banco : GerenciamentoBanco
+ __init__(gerenciamento_banco : GerenciamentoBanco) : void + obter_atividades() : List<atividades> + obter_detalhes_atividade(id_atividade : dict<atividade>) : List<atividades>

-banco

Instância de GerenciamentoBanco, que gerencia conexões e consultas no banco de dados.

Métodos:

+ __init__(gerenciamento_banco: GerenciamentoBanco):

Método construtor que inicializa o atributo banco.

+obter_atividades(): List<atividades>

Executa uma consulta SQL genérica para buscar todas as atividades.

Retorna os resultados obtidos ou None caso ocorra algum erro.

+obter_detalhes_atividade(id_atividade:dict<atividade>) : List<atividades>

Realiza uma consulta SQL específica para buscar os detalhes de uma atividade identificada pelo id_atividade.

Retorna os detalhes obtidos ou None em caso de erro.

CLIENTE
- banco : GerenciamentoBanco
+ __init__(gerenciamento_banco : GerenciamentoBanco) : void + obter_clientes() : List<Cliente> + obter_detalhes_cliente(id_cliente : int) : dict<Cliente> + atualizar_cliente(dados_atualizados : dict<Cliente>, id_cliente : int) : boolean

Atributo:

- banco: GerenciamentoBanco

Atributo privado que armazena uma instância de GerenciamentoBanco.

Métodos:

+ __init__(gerenciamento_banco: GerenciamentoBanco)

Método construtor que inicializa o atributo banco.

+ obter_clientes() : List<Cliente>

Método que recupera todos os clientes e retorna uma lista ou None em caso de erro.

+ obter_detalhes_cliente(id_cliente):dict<Cliente>

Método que recupera os detalhes de um cliente específico com base no id_cliente.

+ atualizar_cliente(dados_atualizados:dict<Cloiente>, id_cliente: int): boolean

Método que atualiza os dados de um cliente específico. Retorna True se bem-sucedido ou False em caso de erro.

ESTOQUE
- banco : GerenciamentoBanco
+ __init__(gerenciamento_banco : GerenciamentoBanco) : void + validar_produto(id_produto : int) : dict<Produtos> + obter_produtos() : List<Produtos> + obter_materia_prima() : dict<MateriaPrima> + obter_detalhes_materia_prima(id_materia : int) : List<MateriaPrima> + excluir_produto(id_produto : int, excluir_estoque : boolean) : boolean

-banco

Representa a instância de GerenciamentoBanco, usada para executar consultas no banco de dados.

Método

+ __init__(gerenciamento_banco: GerenciamentoBanco)

Método construtor que inicializa o atributo banco.

+ validar_produto(id_produto : int) : dict<Produtos>

Verifica se um produto específico existe na tabela Produto com base no id_produto. Retorna os dados do produto ou None se não for encontrado.

+obter_produtos(): List<Produtos>

Realiza uma consulta SQL para obter todos os produtos disponíveis no estoque. Retorna uma lista de produtos ou None em caso de erro.

+obter_materia_prima(): dict<Produtos>

Busca todas as matérias-primas disponíveis no estoque. Retorna uma lista de matérias-primas ou None em caso de erro.

+obter_detalhes_materia_prima(id_materia: int): List<Produtos>

Consulta os detalhes de uma matéria-prima específica identificada por id_materia. Retorna um dicionário com os detalhes ou None em caso de erro.

+ excluir_produto(id_produto : int, excluir_estoque : boolean) : boolean

Exclui um produto da tabela Produto. Se excluir_estoque for True, remove também os dados relacionados na tabela Produtos.

Utiliza commit() para confirmar as alterações no banco.

FORNECEDOR
- banco : GerenciamentoBanco
+ __init__(gerenciamento_banco : GerenciamentoBanco) : void + obter_fornecedores() : List<Fornecedor> + obter_detalhes_fornecedores(id_fornecedor : int) : dict<Fornecedor> + atualizar_fornecedor(dados_atualizados : dict<Fornecedor>, id_fornecedor : int) : boolean

- banco: GerenciamentoBanco

Atributo privado que armazena uma instância de GerenciamentoBanco.

Metodos:

+ __init__(gerenciamento_banco: GerenciamentoBanco)

Método construtor que inicializa o atributo banco.

+obter_fornecedores():List<Fornecedor>

Busca todos os fornecedores cadastrados no banco de dados.

Retorna uma lista de fornecedores ou None em caso de erro.

+obter_detalhes_fornecedores(id_fornecedor:int):dict<fornecedor>

Obtém os detalhes de um fornecedor específico com base no id_fornecedor.

Retorna um dicionário com os detalhes ou None se ocorrer erro.

+atualizar_fornecedor(dados_atualizados:dict<fornecedor>,id_fornecedor:int):boolean

Atualiza as informações de um fornecedor no banco de dados, usando os dados fornecidos no dicionário dados_atualizados.

PEDIDO
- banco : GerenciamentoBanco
+ __init__(gerenciamento_banco : GerenciamentoBanco) : void + obter_pedidos_abertos() : List<pedido> + obter_pedidos_finalizados() : List<pedido> + obter_detalhes_pedido(id_pedido : int) : dict<pedido> + contar_pedidos_abertos() : int

- banco: GerenciamentoBanco

Atributo privado que armazena uma instância de GerenciamentoBanco.

Metodos:

+ __init__(gerenciamento_banco: GerenciamentoBanco)

Método construtor que inicializa o atributo banco.

+obter_pedidos_abertos():List<pedido>

Realiza uma consulta SQL para buscar todos os pedidos que possuem o status 'aberto'.

Retorna uma lista de pedidos abertos ou None em caso de erro.

+obter_pedidos_finalizados():List<pedido>

Realiza uma consulta SQL para buscar todos os pedidos que possuem o status 'finalizado'.

Retorna uma lista de pedidos finalizados ou None em caso de erro.

+obter_detalhes_pedido(id_pedido:int):dict<pedido>

Consulta os detalhes de um pedido específico com base no id_pedido.

Retorna um dicionário com os detalhes do pedido ou None se ocorrer erro.

+contar_pedidos_abertos() : int

FUNCIONARIO	
- banco : GerenciamentoBanco	
- sessao : dict<funcionario>	
+ salvar_sessao(usuario : dict<funcionario>) : void	
+ obter_sessao() : dict<funcionario>	
+ limpar_sessao() : void	
+ obter_funcionarios() : List<FUNCIONARIO>	
+ obter_detalhes_funcionario(id_funcionario : int) : dict<funcionario>	
+ atualizar_funcionario(dados_atualizados : dict<funcionario>, id_funcionario : int) : boolean	
+ obter_funcionari_login() : List<FUNCIONARIO>	

- banco: GerenciamentoBanco

Atributo privado que armazena uma instância de GerenciamentoBanco.

Metodos:

+ salvar_sessao(usuario : dict<funcionario>) : void

Salva as informações do usuário na sessão em um dicionário.

Exibe uma mensagem de confirmação.

+ obter_sessao() : dict<funcionario>

Retorna o dicionário contendo as informações do usuário logado.

+ limpar_sessao() : void

Limpa os dados da sessão, efetivamente deslogando o usuário.

+ obter_funcionarios() : List<funcionario>

Consulta todos os funcionários cadastrados no banco de dados.

Retorna uma lista de funcionários ou None em caso de erro.

+ obter_detalhes_funcionario(id_funcionario : int) : dict<funcionario>

Obtém os detalhes de um funcionário específico identificado pelo id_funcionario.

Retorna um dicionário com os detalhes ou None se ocorrer erro.

+ atualizar_funcionario(dados_atualizados : dict<funcionario>, id_funcionario : int) : boolean

Atualiza as informações de um funcionário no banco de dados com base nos dados fornecidos no dicionário dados_atualizados.

+ obter_funcionari_login() : List<funcionario>

Obtém os IDs, nomes e logins de todos os funcionários.

Retorna uma lista com as informações ou None em caso de erro.

Validação
<pre> + __init__() : void + valid_Login(cpf : str, senha : str, funcionario : FUNCIONARIO) : boolean + verificar_usuario_logado(funcionario : FUNCIONARIO, page : ft.Page) : boolean + Logout(page : ft.Page, funcionario : FUNCIONARIO) : void + @staticmethod() : void + validar_cpf(cpf : int) : boolean </pre>

+ valid_Login(cpf : str, senha : str, funcionario : FUNCIONARIO) : boolean

Recebe o CPF e a senha do usuário.

A senha é criptografada usando SHA-256 antes de ser comparada com a senha armazenada no banco de dados.

Caso o CPF e a senha coincidam com os dados do banco, a sessão do usuário é criada, armazenando suas informações.

Retorna True se o login for válido e False caso contrário.

+ verificar_usuario_logado(funcionario : FUNCIONARIO, page : ft.Page) : boolean

Verifica se o atributo logado no objeto funcionario está definido.

Se não houver usuário logado, redireciona o usuário para a página de login (/login) e retorna False.

Caso contrário, permite o acesso e retorna True.

+ Logout(page : ft.Page, funcionario : FUNCIONARIO) : void

Realiza a limpeza da sessão do usuário, apagando as informações de login.

Redireciona para a página de login (/login).

+@staticmethod() : void

+validar_cpf(cpf:int) : boolean

PRODUÇÃO
<pre> - banco : GerenciamentoBanco + __init__(gerenciamento_banco : GerenciamentoBanco) : void + obter_producao() : List<produção> + obter_detalhes_plantio(id_plantio : int) : dict<produção> + grafico_qnt_prod_mes(quantidade : dict<produção>) : boolean </pre>

- banco: GerenciamentoBanco

Atributo privado que armazena uma instância de GerenciamentoBanco.

Metodos:

+obter_producao(): List<produção>

Consulta todas as produções registradas no banco de dados.

Retorna uma lista de produções ou None em caso de erro.

+obter_detalhes_plantio(id_plantio:int) : dict<produção>

Consulta os detalhes de um plantio específico identificado pelo id_plantio.

Retorna um dicionário com os detalhes do plantio ou None se ocorrer erro.

+grafico_qnt_prod_mes(quantidade:dict<produção>): boolean

Realiza uma consulta para obter a quantidade total de produção por mês.

Excluir
- page : ft.Page - dialog_input : ft.AlertDialog - dialog_confirmacao : ft.AlertDialog - id_input : ft.TextField - nome_input : None - tipo_entidade : str
+ __init__(page : ft.Page) : void + abrir_dialogo_id(tipo_entidade : str) : void + confirmar_exclusao(e : int) : void + excluir_registro(e : int, id_para_excluir : int) : void + fechar_dialogo(e=None : int) : void

- page : ft.Page
- dialog_input : ft.AlertDialog
- dialog_confirmacao : ft.AlertDialog
- id_input : ft.TextField
- nome_input : None
- tipo_entidade : str

Metodos:

+ abrir_dialogo_id(tipo_entidade : str) : void

Solicita ao usuário que insira o ID da entidade a ser excluída.

Configura o tipo da entidade (como "produto" ou "fornecedor") para uso nos métodos seguintes.

+ confirmar_exclusao(e : int) : void

Valida o ID inserido.

Se o ID for inválido ou não encontrado no banco, exibe um *SnackBar* de erro.

Se o ID for válido, abre um segundo diálogo pedindo a confirmação para excluir.

+ excluir_registro(e : int, id_para_excluir : int) : void

Realiza a exclusão do registro no banco de dados com base no tipo de entidade e ID fornecido.

Exibe uma mensagem de sucesso ou erro, dependendo do resultado da exclusão.

+ fechar_dialogo(e=None : int) : void

Fecha quaisquer diálogos abertos, seja de entrada de ID ou confirmação de exclusão.

Cadastro
- page : ft.Page - dialog : ft.AlertDialog - inputs : dict<Cadastro> - senha_input : ft.TextField - dados_salvos : dict<Cadastro>
+ __init__(page : ft.Page) : void + abrir_cadastro(tipo_cadastro : str) : void + abrir_registro(tipo_cadastro : str) : void + _salvar_dados(e : int) : void + inserir_banco() : void + _fechar_dialog(e=None : int) : void + hash_password(password : str) : str + _gerar_campos_formulario(tipo_cadastro : str) : str

- page : ft.Page
- dialog : ft.AlertDialog
- inputs : dict<Cadastro>
- senha_input : ft.TextField
- dados_salvos : dict<Cadastro>

Metodos:

+ abrir_cadastro(tipo_cadastro : str) : void

Abre um diálogo de cadastro com campos definidos dinamicamente de acordo com o tipo de cadastro.

+ abrir_registro(tipo_cadastro : str) : void

Similar ao método anterior, usado para cadastros de registros específicos como pedidos ou produção.

+ _salvar_dados(e : int) : void

Coleta os valores preenchidos pelo usuário, valida e realiza ações adicionais, como a geração do hash de senha.

+ inserir_banco() : void

Insere os dados processados no banco e exibe uma mensagem de sucesso ou erro.

+ _fechar_dialog(e=None : int) : void

Fecha o diálogo de cadastro.

+ hash_password(password : str) : str

Criptografa a senha fornecida usando SHA-256.

+ _gerar_campos_formulario(tipo_cadastro : str) : str

Cria dinamicamente os campos do formulário com base no tipo de cadastro.

Detalhes
- page : Object - dialog : Object - gerenciamento_banco : Object - em_edicao : boolean - botoes : dict - campos : dict - campos_nao_editaveis : List - id_pedido_atual : int - id_atividade_atual : int - id_plantio_atual : int - id_cliente_atual : int - id_fornecedor_atual : int - id_funcionario_atual : int - id_materia_atual : int
+ detalhes_pedido(id_pedido : int) : void + detalhes_atividade(id_atividade : int) : void + detalhes_producao(id_plantio : int) : void + detalhes_cliente(id_cliente : int) : void + detalhes_fornecedor(id_fornecedor : int) : void + detalhes_materia_prima(id_materia : int) : void + detalhes_funcionario(id_funcionario : int) : void + _alternar_modos_edicao(e : object, tipo_entidade : str) : void + salvar_alteracoes(e : object, tipo_entidade : str) : void + _fechar_dialog(e : object = None) : void + hash_password_sha256(password : str) : str

- page : Object
- dialog : Object
- gerenciamento_banco : Object
- em_edicao : boolean

- botoes : dict
- campos : dict
- campos_nao_editaveis : List
- id_pedido_atual : int
- id_atividade_atual : int

- id_plantio_atual : int
- id_cliente_atual : int
- id_fornecedor_atual : int
- id_funcionario_atual : int
- id_materia_atual : int

Metodos:

+ detalhes_pedido(id_pedido : int) : void

Exibe os detalhes de um pedido específico no diálogo.

+ detalhes_atividade(id_atividade : int) : void

Exibe os detalhes de uma atividade específica no diálogo.

+ detalhes_producao(id_plantio : int) : void

Exibe os detalhes de uma produção específica no diálogo.

+ detalhes_cliente(id_cliente : int) : void

Exibe e permite editar os detalhes de um cliente.

+ detalhes_fornecedor(id_fornecedor : int) : void

Exibe e permite editar os detalhes de um fornecedor.

+ detalhes_materia_prima(id_materia : int) : void

Exibe os detalhes de uma matéria-prima.

+ detalhes_funcionario(id_funcionario : int) : void

Exibe e permite editar os detalhes de um funcionário, incluindo senha.

+ _alternar_modos_edicao(e : object, tipo_entidade : str) : void

Altera os campos do diálogo entre os modos de leitura e edição, ajustando o botão de "Salvar".

+ salvar_alteracoes(e : object, tipo_entidade : str) : void

Salva as alterações realizadas, enviando as atualizações ao banco de dados.

+ _fechar_dialog(e : object = None) : void

Fecha o diálogo sem salvar alterações.

+ hash_password_sha256(password : str) : str

Criptografa uma senha usando SHA-256 para armazenar no banco.

ConfirmaçãoDialog
- title : str - content : str - actions : List - page : Object - dialog : Object
+ __init__(title : str, content : str, actions : list, page : object) : void + create_dialog() : Object + open_dialog() : void + close_dialog(e : object = None) : void

- title : str
- content : str
- actions : List
- page : Object
- dialog : Object

Métodos:

+ __init__(title : str, content : str, actions : list, page : object) : void

Inicializa os atributos do diálogo com o título, conteúdo, ações e página fornecidos.

+ create_dialog() : Object

Cria o diálogo configurado com título, conteúdo e ações.

+ open_dialog() : void

Exibe o diálogo na página.

+ close_dialog(e : object = None) : void

Fecha o diálogo; pode receber um evento opcional como parâmetro

ConfirmaçãoSaida
- (inherits ConfirmationDialog) : int
+ __init__(page : object) : void + abrir() : void + sair(e : object = None) : void

Herança: Herda todos os atributos e métodos de **ConfirmationDialog**.

Métodos

+ __init__(page : object) : void

Define valores específicos de título e conteúdo para o caso de saída.

+ abrir() : void

Cria e exibe o diálogo de saída com botões "Sair" e "Cancelar".

+ sair(e : object = None) : void

Define a lógica de redirecionamento para a página de login e fecha o diálogo.

Atualizar
- page : Object - dialog_atualizar : str - rg_input : str - senha_atual_input : str - nova_senha_input : str - confirma_nova_senha_input : str - funcionario : FUNCIONARIO - fechar_dialogo(self, e : int = None)
+ __init__(self : int, page : int) : void + abrir_dialogo_atualizar(self : int) : void + atualizar_senha(self : int, e : int) : void + exibir_snack_bar(self : int, mensagem : int, cor : int) : boolean + fechar_dialogo(self : int, e=None : int) : void + hash_password(self : int, password : int) : str

Page : Object

dialog_atualizar : str

rg_input : str

senha_atual_input : str

nova_senha_input : str

confirma_nova_senha_input : str

funcionario: FUNCIONARIO

Métodos

+__init__(self : int, page : int) : void

Inicializa a classe com a página do aplicativo e configura os atributos para os componentes da interface.

+abrir_dialogo_atualizar(self : int) : void

Exibe um diálogo para o usuário inserir as informações necessárias para atualizar a senha. Configura os campos de entrada e botões de ação (Atualizar e Cancelar).

+atualizar_senha(self : int, e : int) : void

Realiza validações nos campos preenchidos pelo usuário. Verifica se o RG e a senha atual correspondem no banco de dados. Atualiza a senha no banco caso todas as condições sejam atendidas e exibe mensagens de feedback ao usuário.

+exibir_snack_bar(self : int, mensagem : int, cor : int) : boolean

Mostra uma notificação (snackbar) com uma mensagem e uma cor de fundo para fornecer feedback ao usuário.

+fechar_dialogo(self : int, e=None : int) : void

Fecha o diálogo de redefinição de senha.

+hash_password(self : int, password : int) : str

Criptografa a senha para armazenamento seguro no banco de dados (implementação ausente).